API BEST PRACTICES



MA PRIMA...



OPIS: FA33KJF7



CAMPI OPZIONALI

In OpenAPI, il campo `required` definisce le regole di validazione del *payload* e non il comportamento logico dell'API.



I. REQUIRED

La chiave `required` in uno schema determina se un campo **deve essere presente** nel payload **in ingresso** (ad esempio, nel `requestBody` di una `POST` o `PUT`).

Contesto	Campo `id` (`readOnly: true`)	Regola di `required`
`POST /fountains`	L'ID non può essere inviato.	**Deve essere omesso** nel `requestBody`.
`PUT /fountains/{id}`	L'ID non deve essere modificato dal client.	**Deve essere inviato** (per la sostituzione completa) ma il server lo ignora/valida.
`GET /fountains/{id}`	L'ID è restituito.	**Deve essere presente** nello schema di risposta.

☐

2. ESEMPI E REQUIRED

I campi opzionali (quelli non elencati in `required: []`) e i campi in sola lettura (`readOnly: true`) dovrebbero essere gestiti in modo specifico negli esempi (come quelli visibili in Swagger UI):

- **Esempio di `requestBody` (POST):**
  - L'esempio **NON DEVE** includere il campo `id`.
  - *Obiettivo:* Guidare lo sviluppatore client a inviare il payload minimo e corretto.
- **Esempio di `responseBody` (201 Created):**
  - L'esempio **DEVE** includere il campo `id`.
  - *Obiettivo:* Mostrare al client la struttura completa della risorsa creata.



ESEMPIO: FOUNTAIN SCHEMA E USO

Definizione dello Schema (in `components/schemas/Fountain`):

```yaml
Fountain:
  type: object
  properties:
    id:
      $ref: '#/components/schemas/FountainIdSchema'
    state:
      type: string
  required: # State è richiesto, l'ID no (è generato)
    - state
```



Uso nell'Operazione `POST /fountains`:

L'esempio per la richiesta (`requestBody`) dovrebbe riflettere i campi richiesti (`state`, `latitude`, `longitude`) ma **escludere** l'ID:



- [ ]

API BEST PRACTICES



URI - SOSTANTIVI, NON VERBI

- Principio: URI Rappresentano Risorse, Non Azioni.
- **USA** i **sostantivi** (`nouns`) nelle URI, poiché le URI identificano oggetti (risorse).
- **MAI** includere **verbi** (`verbs`) nell'URI della risorsa. L'azione (cosa fare) è definita dal **Metodo HTTP**.



Metodo	Esempio (Corretto)	Esempio (Errato)
`GET`	`/managed-devices/{id}`	`/get-managed-device/{id}`
`PUT`	`/managed-devices/{id}`	`/update-managed-device/{id}`
`POST`	`/users`	`/create-user`
`DELETE`	`/managed-devices/{id}`	`/remove-managed-device/{id}`

- **Convenzione:** Usa sostantivi **singolari** per le singole risorse e **plurali** per le collezioni.
  - Singolo: `/users/admin`
  - Collezione: `/users`

☐

STRUTTURA E GERARCHIA



PRINCIPIO: GERARCHIA LOGICA E SEPARATORI.

- **Gerarchia:** Usa lo slash (`/`) per esprimere una **gerarchia logica** nelle relazioni tra le risorse. Questo aiuta l'API a crescere in modo ordinato.
- **Separatore:** Lo slash finale (`/`) è comunemente usato per indirizzare una collezione di risorse (es. `/users/`).

Esempio di Annidamento Logico:

```
/users/{userid}/devices/{deviceid}
```

(Il dispositivo `{deviceid}` è una risorsa contenuta all'interno della collezione di dispositivi dell'utente `{userid}`).

- [ ]

GESTIONE DEGLI ERRORI E FILTRI

I. CODICI DI STATO HTTP

- **REST ≠ OK/Errore:** Molti errori possono essere mappati direttamente a specifici codici di stato HTTP.
  - Parametri mancanti o valori non validi: 400 **Bad Request**.
  - Risorsa non trovata: 404 **Not Found**.
  - Successo: 200 **OK**, 201 **Created**, 204 **No Content**.
- **Corpo di Errore:** Se necessario, fornisci dettagli aggiuntivi (per gli sviluppatori) in un corpo di risposta comune (es. JSON con campi specifici sull'errore).



2. QUERY PER FILTRI E PAGINAZIONE

- **USA** i componenti **query string** della URI per filtrare e impaginare i risultati.
- **MAI** usare il corpo (`body`) o l'URI del percorso (`path`) per i filtri. La collezione è unica, i parametri la limitano.

Esempio:

```
/managed-devices/?region=USA
```



DATI BINARI E CORPO DELLA RICHIESTA



DATI BINARI (FILE/STREAM)

- **Evita l'embedding in JSON:** Non è consigliabile incorporare grandi dati binari (es. foto) direttamente all'interno di JSON per motivi di performance.
- **Strategia Consigliata:**
  1. **Invio (Client → Server):**
     - Usa `multipart/form-data` per inviare dati strutturati (JSON) e il file in un'unica richiesta. **OPPURE**
     - Carica il file in una API dedicata (`POST /images/`) e poi usa l'URL/ID dell'immagine nella richiesta JSON principale.
  2. **Ricezione (Server → Client):**
     - Restituisci un JSON con l'informazione strutturata e l'**URL completo** per accedere al file binario in un endpoint separato (es. `/images/{identifier}`).

A. Upload di dati strutturati + file (multipart)

Se devi inviare dati JSON insieme a un file binario, l'approccio standard è utilizzare `multipart/form-data`.
Scenario: Aggiungere una nuova fontana (JSON) e caricarne contemporaneamente la foto (File).

Corpo della Richiesta (Immagine logica):

- **Media Type:** `multipart/form-data`
- **Part 1 (Dati JSON):** `Content-Disposition: name="fountain_data"` (Contiene il JSON `{ "state": "good", ... }`)
- **Part 2 (File Binario):** `Content-Disposition: name="photo"` (Contiene i byte binari dell'immagine)

☐

Quando il server risponde, deve evitare di inviare direttamente il contenuto binario all'interno del JSON.

I. RISPOSTA DIRETTA DEL FILE

Se un endpoint serve **solo** il file, la risposta non è JSON:

```
GET /images/fountain_123.jpg
```

Risposta HTTP:

- **Status:** 200 OK
- **Content-Type:** `image/jpeg` (o `image/png`, ecc.)
- **Body:** Contiene i byte grezzi (binari) dell'immagine.

2. REFERENZIAMENTO NEL JSON

Quando si listano le fontane o si recupera una singola risorsa, il JSON deve contenere un riferimento (un URL o un ID) al file.

Esempio di Risposta JSON (Referenziamento):

```
{
  "id": 1,
  "state": "good",
  "latitude": 41.89025,
  "longitude": 12.49237,
  "photo_url": "/api/v1/images/fountain_123.jpg"  // Riferimento al file
}
```

Vantaggio: Il client decide se e quando scaricare l'immagine, ottimizzando i tempi di caricamento della lista principale.

ESEMPIO CON CURL

Il comando `curl` semplifica enormemente l'uso di `multipart/form-data` grazie all'opzione `-F` (o `--form`).

Quando si usa `-F`, `curl` esegue automaticamente tre azioni cruciali:

1. Imposta l'intestazione `Content-Type: multipart/form-data`.
2. Genera e imposta un **valore di `boundary` unico** per la richiesta.
3. Formatta il corpo della richiesta, separando correttamente i dati.

Scenario Pratico: Aggiungere una fontana (`JSON`) e caricarne la foto (`File`).



Struttura della Richiesta:

Il comando `curl` richiede un parametro `-F` per ogni "parte" che deve essere inviata.

arte	Opzione cURL	Descrizione
Dati Strutturati JSON)	`-F "fountain_data=@data.json;type=application/json"`	Carica il contenuto di `data.json` e ne specifica il `Content-Type`.
File Binario Immagine)	`-F "photo=@image.jpg;type=image/jpeg"`	Carica il file binario `image.jpg`.

- [ ]

1. Preparazione del File JSON (`data.json`): Creiamo un file JSON che rappresenta i dati strutturati della nuova fontana (senza l'ID, generato dal server).

```
{
  "state": "good",
  "latitude": 41.89025,
  "longitude": 12.49237
}
```

2. Comando cURL Completo:

```
curl -X POST https://api.nasoniroma.it/v1/fountains \
     -F "fountain_data=@data.json;type=application/json" \
     -F "photo=@/percorso/alla/tua/foto.jpg;type=image/jpeg"
```

Risultato: `curl` crea una singola richiesta HTTP POST, contenente al suo interno sia i dati JSON che i byte dell'immagine, pronti per essere elaborati dal server.

2. RESTRIZIONI SUL CORPO

- **GET e DELETE: Non definire un corpo** nelle richieste `GET` e `DELETE`. Sebbene `DELETE` possa tecnicamente avere un corpo (per lo standard HTTP), è sconsigliato in REST per evitare comportamenti indefiniti e complicazioni con l'idempotenza.



GESTIONE DELLE SOTTO-COLLEZIONI

PRINCIPIO: NON SCAMBIARE NOME DELLA COLLEZIONE E ELEMENTO

- **Il Problema:** Se una risorsa (o un'azione) ha effetto **solo sull'utente autenticato** (es. "mettere a tacere Bob"), non annidarla direttamente sotto la risorsa che rappresenta l'entità target ("Bob").
- **Esempio Sbagliato (Non Idempotente):**

```
/users/{userid}/mute
# Sembra che tu stia mutando l'utente {userid} per tutti.
```



- **Soluzione Corretta (Collezione Relazionale):** Tratta l'azione come una **collezione di risorse** relative all'utente autenticato.
    - **Io sto mutando qualcuno:** `muted` (gente che ho messo a tacere) è una collezione.

```
/me/muted/{mutedid}
# La risorsa {mutedid} fa parte della mia collezione personale /me/muted.
```

    - **POST/PUT** a questo percorso: Mette a tacere un utente.
    - **DELETE** a questo percorso: Toglie il muto all'utente.

Questo approccio garantisce che le azioni siano **idempotenti**, che l'operazione di "unmuting" sia nativamente gestita dal `DELETE`, e che la risorsa sia chiaramente **per-utente-autenticato**.



AGGIORNAMENTO DELLE RISORSE: PUT VS. PATCH

Quando si tratta di modificare una risorsa esistente, si usano principalmente due metodi HTTP con semantiche molto diverse.



PUT (SOSTITUZIONE COMPLETA)

Caratteristica	Descrizione
Scopo	**Sostituire** la risorsa bersaglio con il contenuto completo inviato nel corpo della richiesta.
Idempotenza	**Sì**. Eseguire la stessa richiesta PUT più volte produce lo stesso stato finale sul server.
Dati richiesti	L'**intera** rappresentazione della risorsa. Se ometti un campo, questo dovrebbe essere eliminato o azzerato dal server.
Tipico Uso	Modifiche complete, come la riscrittura di un documento.



Esempio (Aggiornamento Fontana - PUT):

Il client invia l'intera risorsa, anche se ha modificato solo lo stato:

```
{
  "id": 1,
  "state": "faulty",    // Modificato
  "latitude": 41.89025, // Non modificato, ma deve essere inviato
  "longitude": 12.49237 // Non modificato, ma deve essere inviato
}
```



PATCH (AGGIORNAMENTO PARZIALE)

Caratteristica	Descrizione
Scopo	Applicare una serie di **modifiche parziali** alla risorsa bersaglio.
Idempotenza	**No**, non intrinsecamente. Una richiesta `PATCH` è idempotente solo se il server la implementa per esserlo.
Dati richiesti	**Solo i campi da modificare**. I campi omessi rimangono invariati.
Tipico Uso	Modifiche mirate, come cambiare solo lo stato o il flag di una risorsa (es. mutare un utente, cambiare stato di una fontana).



Esempio (Aggiornamento Fontana - PATCH):

Il client invia **solo** il campo modificato, riducendo il traffico:



ESEMPIO OPENAPI PER PATCH

```yaml
/fountains/{id}:
  patch:
    operationId: updateFountainState
    summary: Segnala un guasto o correggi le coordinate
    description: |
      Applica aggiornamenti parziali ai campi forniti.
    requestBody:
      description: |
          Proprietà della fontana da aggiornare
          (solo i campi specificati vengono modificati)
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              state:
                $ref: '#/components/schemas/Fountain/properties/state'
              latitude:
                $ref: '#/components/schemas/Fountain/properties/latitude'
              longitude:
                $ref: '#/components/schemas/Fountain/properties/longitude'
```

```
      responses:
        '200':
          description: Aggiornamento applicato con successo
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Fountain'
        '404':
          description: Fontana non trovata
```

